

计算机网络实验报告一

基于Socket实现本地实时多人聊天室程序

学号:2013365 姓名:颜铭

1 实验准备

1. 套接字通信

套接字 (Socket) 是实现网络通信的基本工具, 它允许计算机之间通过网络进行数据传输。在实验之前, 需要掌握套接字编程, 了解套接字的创建、绑定、监听、接受连接以及发送和接收数据的操作

套接字编程的实现是在网络层, 涉及到网卡驱动和系统资源调度等封闭和安全性问题, 因此不同操作系统的 socket 通信标准库并不一致也不完全兼容。本次实验在 windows 系统上进行实验使用的是 Win32 标准的 inet VC 风格的库 `WinSock2`, 其详细文档见[官网手册](#)

Winsock2 遵循了 Windows 开放系统架构 (Windows Open System Architecture, WOSA) 模型, 它在应用程序编程接口 (API) 与协议栈之间定义了一个标准的服务提供者接口 (SPI)。该模型采用了套接字范式, 最早是由伯克利软件发行 (BSD) UNIX 广泛使用的。后来, 在 Windows Sockets 1.1 版本中, 它被适配到了 Windows 平台, 使得 Windows Sockets 2 应用程序具备向后兼容性。Winsock 编程以前主要围绕 TCP/IP 展开

其教程初始化引入代码(需要连接静态库)

```
1 #include <WS2tcpip.h>
2 #include<WinSock2.h>
3 #pragma comment (lib, "ws2_32.lib")
4
```

2. 多线程编程

线程(thread)是一个进程(process)内的操作系统最小资源可执行单元。作为进程内的一个独立执行流, 共享进程的资源, 包括内存空间和文件句柄。线程是现代操作系统实现多任务并发执行的关键组成部分, 使得程序能够同时执行多个任务, 提高了计算机的效率和响应性。

由于多人聊天室要允许服务器同时对多人的收发流式信息的监听，是一个高并发的场景，需要使用互斥锁和条件量甚至是并行来解决通信的可能存在的延迟问题。线程执行必须和主线程分离开并及时销毁回收而防止造成无响应的状态，所以一般对接受信息 `recv()` 函数进行多线程分离并等待 `join()` 调度，再分别根据请求解析出回应分发 `send()` 即可。

3. 学习tcp协议,并基于Socket通信自定义协议请求和响应实现功能

手动实现tcp/ip完备的协议栈是困难的，我们本次实验只基于socket api模拟主流的通信流程响应系统内置的tcp识别状态。

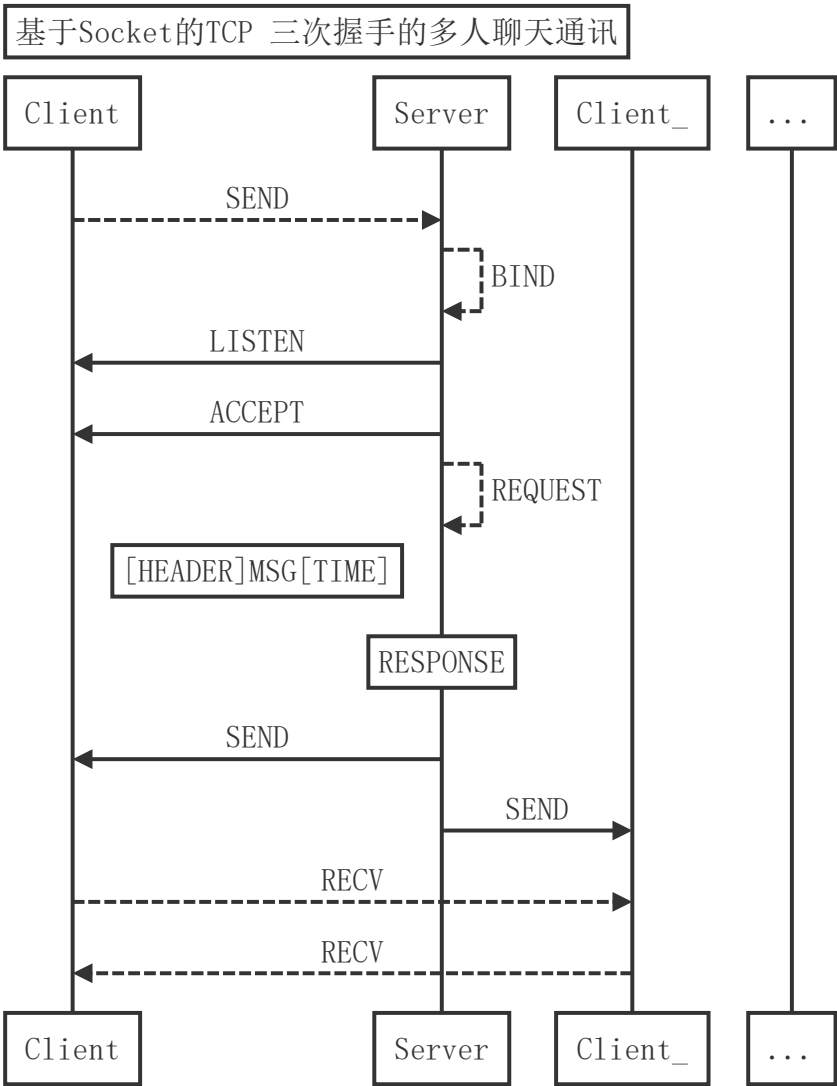
具体地我们的协议传输文本具有如下形式

`PROTOCOL`:

[HEADER]MSG[TIME]

其中HEADER是表头，具体类别见第二节。MSG是分发文本信息，其中包含对MSG的校验和进行计算来验证是否有信息损失。并通过时间戳记录时间

本次基于套接字的多人聊天通信模拟tcp协议流程示意图如下



4. 测试和调试

由于本地端无法方便地模拟抖动等通信质量问题，不需要特别的抓包测试。可以作为后续实验跟进的一部分拓展内容。

此外，本次实验没有从安全性方面进行多的测试。在编程实现中只通过了 `assert` 进行条件检查防止异常，没有进一步对无法处理的中断(未发现)进行处理。

2 协议设计

2.1 消息等级和日志系统:

首先我们要设计消息等级类型，以方便日志进行打印（日志系统记录文本进行后台管理）

消息类型实现:

```
1
2 enum CliMessageType {
3     MSG_TYPE_UNKNOWN = 0, // 丢失消息的未知数据
4     MSG_TYPE_LOGIN, // 登入
5     MSG_TYPE_CALL, // 消息调用, 撤回请求
6     MSG_TYPE_CHAT, // 正常通讯聊天
7     MSG_TYPE_GET, // 消息回溯, 历史请求
8     MSG_TYPE_LOGOUT // 登出
9 };
```

我们自定义的协议为

[HEADER]MSG[TIME]

这里的HEADER就对应以上的

- UNKNOWN: 消息丢失
- LOGIN: 登入消息
- CALL: 撤回消息
- CHAT: 正常聊天
- GET: 历史消息
- LOGOUT: 登出消息

服务器此外还会在登入和登出时进行广播

同理我们设置必要的日志类进行对不同协议用户消息体的解析如下:

```
1 class TcpNwLogger
2 {
3 private:
4     std::string signature = "To Hash: Protect Safety For User";
5     std::time_t timestamp;
6     bool isClosed;
7 protected:
8     char* buffer;
9 }
```

```

10     std::fstream logFile;
11     std::FILE* f_id;
12
13 public:
14     TcpNwLogger(std::string filename);
15     ~TcpNwLogger();
16     std::string computeSign(const std::string& data);
17     void Log(const ClientsMsg& CliMsg);
18 };
19
20

```

其最重要的 `Log()` 函数设计如下:

```

1 void TcpNwLogger::Log(const ClientsMsg& CliMsg)
2 {
3
4     /*
5      * 自定义日志系统实现
6      */
7     if (!isClosed)
8     {
9         std::time_t now = std::time(0);
10        std::tm* currentTime = std::localtime(&now);
11        char timestamp[20];
12        std::strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", currentTime);
13
14        int uid;
15        std::string msg_special;
16
17        switch (CliMsg.msgType) {
18            case CliMessageType::MSG_TYPE_LOGIN:
19                std::cout << "[" << "LOGIN" << "]" " << CliMsg.name << ",Connect!!! [" << timestamp
20                << "]" " << std::endl;
21                logFile << "[" << "LOGIN" << "]" " << CliMsg.name << ",Welcome To Connect!!! [" <<
22                timestamp << "]" " << std::endl;
23                break;
24            case CliMessageType::MSG_TYPE_LOGOUT:
25                std::cout << "[" << "LOGOUT" << "]" " << CliMsg.name << ",GoodBye!!! [" << timestamp
26                << "]" " << std::endl;
27                logFile << "[" << "LOGOUT" << "]" " << CliMsg.name << ",GoodBye!!! [" << timestamp
28                << "]" " << std::endl;
29                break;
30            case CliMessageType::MSG_TYPE_CHAT:

```

```

27         std::cout << "[" << "CHAT ONLINE" << "]" " << CliMsg.msg_data << " [" << timestamp
<< "]" " << "(" << CliMsg.name << ")" << std::endl;
28         logFile << "[" << "CHAT ONLINE" << "]" " << CliMsg.msg_data << " [" << timestamp <<
" ] " << "(" << CliMsg.name << ")" << std::endl;
29         break;
30         case CliMessageType::MSG_TYPE_CALL:
31             uid = CliMsg.u_id - BASE_INDEX_CODE;
32             msg_special = history_msg[CliMsg.name][chatCount[uid] - calledIdx[uid]];
33             std::cout << "[" << "CALL" << "]" " << msg_special << " [" << timestamp << "]" " << "
(" << CliMsg.name << ")" << std::endl;
34             logFile << "[" << "CALL" << "]" " << msg_special << " [" << timestamp << "]" " << "("
<< CliMsg.name << ")" << std::endl;
35             break;
36         case CliMessageType::MSG_TYPE_GET:
37             uid = CliMsg.u_id - BASE_INDEX_CODE;
38             msg_special = history_msg[CliMsg.name][chatCount[uid] - calledIdx[uid]];
39             std::cout << "[" << "GET" << "]" " << msg_special << " [" << timestamp << "]" " << "
(" << CliMsg.name << ")" << std::endl;
40             logFile << "[" << "GET" << "]" " << msg_special << " [" << timestamp << "]" " << "("
<< CliMsg.name << ")" << std::endl;
41             break;
42         case CliMessageType::MSG_TYPE_UNKNOWN:
43             msg_special = "Unknown Received Msg";
44             std::cout << "[" << "UNKNOWN" << "]" " << "message loss!!!: "<<msg_special << " ["
<< timestamp << "]" " << "(" << CliMsg.name << ")" << std::endl;
45             logFile << "[" << "UNKNOWN" << "]" " << "message loss!!!: "<<msg_special << " [" <<
timestamp << "]" " << "(" << CliMsg.name << ")" << std::endl;
46             break;
47         default:
48             std::cerr << "something wrong,shutdown?" << std::endl;
49             break;
50     }
51 }
52 }

```

2.2 协议用户体设计

我们的用户体包含了时序逻辑，以及消息主体，以及请求表头(函数语法)，此外补充了校验和进行对消息 **UNKNOWN** 状态的检查，防止信息丢失。

```

1  inline struct ClientsMsg
2  {
3      int u_id = 0;
4      SOCKET soc;

```

```

5     std::string name; //用户名
6     std::string address = "127.0.0.1"; //用户地址(没有网络端解析默认本地)
7     std::string routing; //用户路由(未使用)
8
9
10    std::time_t login_T;
11    std::time_t logout_T;
12    std::string login_time="0"; //登录时间(login time)字串,[年-月-日 时:分:秒]
13    std::string logout_time = "-1"; //登出时间(logout time)字串
14
15    std::string msg_data;
16    unsigned long checksum; //校验和
17    CliMessageType msgType; //消息等级
18    bool isOnline;
19 }Sc[MAX_CLIENT_NUM];
20

```

2.3 服务器类和用户端实现：

我们要通过服务器的Socket进行统一调度，包含创建到主线程的运行 `run()`

```

1  class TCPServer {
2      /*
3      * TCP 服务器端
4      */
5
6  public:
7      TCPServer();
8      TCPServer(std::string ipAddress, int port);
9      ~TCPServer();
10
11     void sendMsg(int clientSocket, std::string msg);
12     bool initWinsock();
13     void run();
14     void cleanupWinsock();
15
16
17 private:
18     SOCKET createSocket();
19     std::string listenerIPAddress;
20     int listenerPort;
21     //MessageReceivedHandler messageReceived;
22 };
23

```

用户端要保存对象名字，且主线程发和多线程接受以防止冲突如下：

这样我们在调用服务器主线程时最大程度避免IO阻塞和未复用对多线程的干扰

```
1  class TCPClient
2  {
3      /*
4       * TCP 用户端
5       */
6
7
8  public:
9
10     TCPClient();
11     ~TCPClient();
12     bool initWinsock();
13     void connectSock();
14     void sendMsg(std::string txt);
15     std::thread recvThread;
16     void threadRecv();
17     std::string username;
18     bool joinChat = true;
19
20
21 private:
22     SOCKET createSocket();
23     std::string serverIP = "127.0.0.1";
24     int serverPort = 54010;
25     sockaddr_in hint;
26     SOCKET serverSocket;          //This is the socket we will connect to.
27     bool recvThreadRunning;
28
29
30 };
```

3 模块封装

3.1 基础通讯

3.1.0.1 bind->listen->accept

首先我们要建立连接并完成对用户结构体初始化，其封装实现为：

1. BIND和LISTEN的套接字通讯准备

```
1  SOCKET TCPServer::createSocket() {
2
3      SOCKET listeningSocket = socket(AF_INET, SOCK_STREAM, 0);    //IPv4解析协议
4
5      if (listeningSocket != INVALID_SOCKET) {
6
7          sockaddr_in hint;        //. 用于将IP地址和端口绑定到特定套接字的结构
8          hint.sin_family = AF_INET;    //告诉提示，我们是IPv4地址
9          hint.sin_port = htons(listenerPort);    //告诉提示我们正在使用哪个端口。
10         inet_pton(AF_INET, listenerIPAddress.c_str(), &hint.sin_addr); // 将IP字符串转换为字节
            并传递给我们的提示且Sin_addr是缓冲区。
11
12         int bindCheck = bind(listeningSocket, (sockaddr*)&hint, sizeof(hint)); // 将
            listeningSocket绑定到提示结构。我们告诉它使用什么IP地址族和端口。
13
14         if (bindCheck != SOCKET_ERROR) {                //如果绑定通过
15
16             int listenCheck = listen(listeningSocket, SOMAXCONN);    //告诉套接字是用于监听的。
17             if (listenCheck == SOCKET_ERROR) {
18                 return -1;
19             }
20         }
21
22         else {
23             return -1;
24         }
25
26         return listeningSocket;
27
28     }
```

再对 `accept` 开启主线程持续运转进行用户体的初始化，这里使用了

`std::map<std::string, std::vector<std::string>>history_msg;` 本地消息存储所有用户的历史转存消息文本，并协同到Sc用户体数组中

我们有初始化为：

```
1  while (true) {
2
```



```

3 fd_set copy = master;    //创建新的文件描述符bc, 文件描述符每次都会被销毁。
4 int socketCount = select(0, &copy, nullptr, nullptr, nullptr);    //确定套接字的状态并
    返回正在"工作"的套接字。
5
6 for (int i = 0; i < socketCount; i++) {    //服务器只能接受连接和接收来自客户端的消息。
7     SOCKET sock = copy.fd_array[i];    //循环遍历文件描述符中标识为非侦听的收发对象
    对应的所有套接字。
8
9     if (sock == listeningSocket) {    //Case 1: accept new connection.
10
11         SOCKET client = accept(listeningSocket, nullptr, nullptr);    //接受传入的连接并将其识
    别为新客户端。
12         FD_SET(client, &master);    //向套接字列表中添加新连接。
13         std::string welcomeMsg = "Welcome to YanMing's MultiThread Chat App(欢迎来到多人聊天
    室)\n";
14         std::cout << "New user joined the chat." << std::endl;    //在服务器端记录连接。
15         recv(client, buf, MAX_BUFFER_SIZE, 0);
16         std::string bufString(buf);
17         //std::cout << "recv " << bufString << std::endl;
18         register_info = bufString;
19         std::regex_search(bufString, loginMatch, loginParser);
20
21         auto user_name = loginMatch.str(1);
22
23         Sc[user_count].name = user_name;
24         Sc[user_count].msgType = CliMessageType::MSG_TYPE_LOGIN;
25         std::time_t now = std::time(0);
26         Sc[user_count].login_T = now;
27         std::tm* currentTime = std::localtime(&now);
28         char timestamp[20];
29         std::strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", currentTime);
30         Sc[user_count].login_time =static_cast<std::string>(timestamp);
31         Sc[user_count].u_id = BASE_INDEX_CODE + user_count;
32         Sc[user_count].isOnLine = true;
33         Sc[user_count].soc = client;
34
35         logger.Log(Sc[user_count]);
36         user_base[user_name] = true;
37         user_count += 1;
38         new_login = true;
39         std::cout << "Current Online User Count is (当前在线有用户总人数) " << user_count <<
    std::endl;
40
41
42     }

```

需要说明的是:

响应消息每次发放的形式都会由程序自动加上用户信息和使用CRC32计算校验和,中间再包括原始的协议文本信息

[HEADER]MSG[TIME] 其中 [HEADER]在正常消息转发时候是可以缺省的。

因此一个完整的响应协议消息是如下形式的:

(USERNAME)[HEADER]MSG(CHECKSUM)[TIME]

这里的CHECKSUM指的就是校验和,以及用户名USERNAME

附:-CRC32的计算校验和原理:

在对信息的处理过程中,我们可以将要被处理的数据块M看成一个n阶的二进制多项式,其形式如下:

$$M = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \dots + a_1x^1 + a_0 \quad (1)$$

而CRC32采用的是最高次为32次幂的多项式,表示如下:

$$X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + \dots + X^1 + 1 \quad (2)$$

校验和(checksum)传输原理如图:



将长度为 k 位的数据块对应一个GF(2)多项式M,以 8 位数据块11100110举例,如果先传输MSB (Most Significant Bit),则它对应的多项式为 $x^7 + x^6 + x^5 + x^2 + x$ (8位对应x的7次幂,因为从x0开始计数,2进制为1时有效)。发送端和接收端约定一个次数为 r 的CRC多项式,取CRC-4 为例: $x^4 + x + 1, r=4$ 。在数据块后面加上r个0对应的多项式为M',显然有 $M' = Mx^r$ 。用 M' 除以CRC-32 将得到一个次数等于或小于 r-1 的余数多项式 R,其对应的 r 位数值则为校验码。发送方通过指定的CRC多项式产生r位的CRC校验码,接收方则通过该CRC多项式来验证收到的报文码的CRC校验码是否为0。

我们使用mod(2)除法:

$$GF(2) = \frac{C(x) \times 2^r}{G(x)} \quad (3)$$

得到的余数R就是校验码,发送的CRC-32编码形式上为 $C(X) \times 2^r + R$

此外，对于三种常用的日志分发 `send` 消息分别准备了不同的请求消息解析器：

```
1 std::regex requestPattern(R"^\(([A-Z]+\))\((\d+)\)"); // 解析请求信息
2 std::regex userParser(R"^\(([a-zA-Z]+\))\((\d+)\)"); //解析用户响应署名信息
3 std::regex loginParser(R"^\(([a-zA-Z]+) joined the chat\)"); //解析登录信息正则
```

3.2 附加请求和响应功能

在基础通信功能上，针对协议表头设计的 `[GET]`（得到历史发送消息）和 `[CALL]`（撤回历史消息）的请求状态，需要主线程 `run` 中实现对请求的响应，其实现代码分别为：

1. 对于历史消息的查询(`GET`):

```
1 auto reqCommand = reqMatch.str(1); // 解析请求的头部
2 if (reqCommand == "GET")
3 {
4     Sc[c].msgType = CliMessageType::MSG_TYPE_GET;
5
6     opt = std::stoi(reqMatch.str(2));
7
8     //std::cout << "debug: " << opt << std::endl;
9     if (opt > history_msg[Sc[c].name].size())
10    {
11        continue;
12    }
13    get_flag = true;
14    get_index = c;
15    recvMsg = history_msg[Sc[c].name][history_msg[Sc[c].name].size() - opt];
16    // 接受到消息处理后返回
17    Sc[c].msg_data = m_data;
18
19    history_msg[Sc[c].name].erase(history_msg[Sc[c].name].end() - opt);
20    logger.Log(Sc[c]);
21 }
```

并且通过 `get_flag` 开关触发对不同套接字对象的消息响应和分发：

```
1 if (outSock != listeningSocket) {
2
3     if (outSock == sock) { //如果当前套接字是发送消息的套接字:
4
5         std::cout << "Message Delivered From " << userName << "\n";
6         // //标记信息被成功从某个用户处发送
7     }
8     else { //如果当前sock不是发送者->它应该接收msg。
9         local_getMsg = userName + ": " + local_getMsg;
```

```

10     send(outSock, local_getMsg.c_str(), strlen (local_getMsg.data()), 0);
11     //向当前套接字发送消息。
12 }
13
14 }
15 if (outSock == listeningSocket)
16 {
17     recvMsg = "聊天记录前" + std::to_string(opt)+"条:"+ recvMsg;
18     send(outSock, recvMsg.c_str(), strlen(recvMsg.data()), 0);
19
20     assert(get_index != -1);
21
22     send(Sc[get_index].soc, recvMsg.c_str(), strlen(recvMsg.data()), 0);
23
24 }
25

```

2. 对于消息的撤回，实现原理是使用 `map<vector>>` 和 `stack` 后进先出记录后按特定的行数进行刷新后重新绘制。因此对需要加密的消息使用 `*` 进行加密即可。

其中刷新终端界面的命令是使用ANSI的控制字符 `\033[%dA\033[K` 这里 `%d` 指的是从当前行开始向上刷新的行数。

```

1  #define ANSI_CLEAR_LINES_TEMPLATE "\033[%dA\033[K"
2
3  // 宏函数，接受一个整数参数并生成相应的控制序列
4  #define ANSI_CLEAR_LINES_FORMAL(numLines) do { \
5      char ansiClearLines[20]; \
6      sprintf_s(ansiClearLines, ANSI_CLEAR_LINES_TEMPLATE, numLines); \
7      printf("%s", ansiClearLines); \
8  } while (0)
9
10

```

我们的实现为:使用全局引用计数器实时维护当期行数，并对响应消息进行解析得到用户名后更新本地对行数内容的日志器即可

```

1  if (header_match == "CALL") {
2
3      auto num_match = stoi(recvParser.str(2));
4
5      auto name_match = recvParser.str(3);
6
7      size_t nameOpt_len = user_responseLine[name_match].size();
8
9      int line_idx = user_responseLine[name_match].at(nameOpt_len - num_match);

```

```

10
11
12
13     line_idx = Line_cnt - line_idx;
14
15     int c = 0;
16     stack<string>().swap(tmpLineMsg);
17     while(c<line_idx)
18     {
19         auto msg = user_responseFlush.top();
20
21
22         user_responseFlush.pop();
23         if (c==line_idx-1)
24         {
25             auto pos = msg.find(name_match);
26             size_t spos = pos + name_match.length();
27
28
29             msg.replace(spos+2, string::npos, msg.size() - spos-3, '*');
30             msg += " (BACK:已撤回)";
31
32         }
33         tmpLineMsg.push(msg);
34         c++;
35     }
36     for(c=1;c<=line_idx;c++)
37         ANSI_CLEAR_LINES_FORMAL(1);
38
39     this_thread::sleep_for(chrono::seconds(1));
40
41     while(!tmpLineMsg.empty())
42     {
43         auto msgRear = tmpLineMsg.top();
44         cout << msgRear << "\n" << std::flush;
45         user_responseFlush.push(msgRear);
46         tmpLineMsg.pop();
47     }
48
49 }

```

3.3 用户自定义退出

类似于请求，我们要手动使用 `[OUT]` 命令，防止 `ctrl+c` 的意外中断导致服务器的无法正确清空聊天数据处理退出的标记

这样我们有服务器的退出操作为：

```

1  else if(reqCommand=="LOGOUT")
2  {
3      Sc[c].msgType = CliMessageType::MSG_TYPE_LOGOUT;
4      std::time_t now = time(0);
5      std::tm*endTime = localtime(&now);
6      char timestamp[30];
7      strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S",endTime);
8      Sc[c].logout_T = now;
9      Sc[c].logout_time = static_cast<std::string>(timestamp);
10
11     auto time_diff = Sc[c].logout_T - Sc[c].login_T;
12
13     auto tdiff_str = std::to_string(time_diff);
14     ;                               Sc[c].msg_data = "OFFLINE;BYE EVERYONE(From Server)[ONLINE
    TIME: "
15         + tdiff_str+ " s]";
16     recvMsg = Sc[c].msg_data;
17     Sc[c].isOnLine = false;
18     history_msg[Sc[c].name].clear();
19     logger.Log(Sc[c]);
20     user_count--;
21 }
22 }

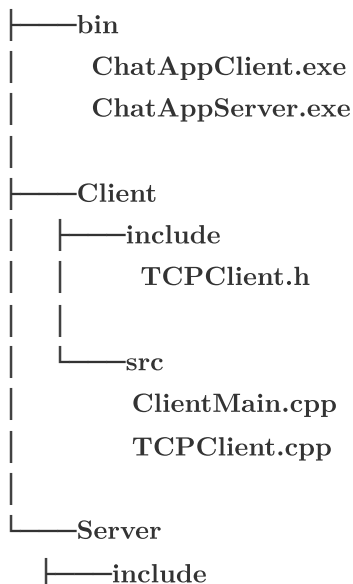
```

同时广播其他用户，给出用户在线时间 $T_{out} - T_{in}$ 的日志，最后释放内存，回收套接字绑定的资源，清空存储的历史消息。

4 实验演示

实验构建采用 `msbuild`, 省略管理解决方案资源文件 `.sln`

我们有文件结构为



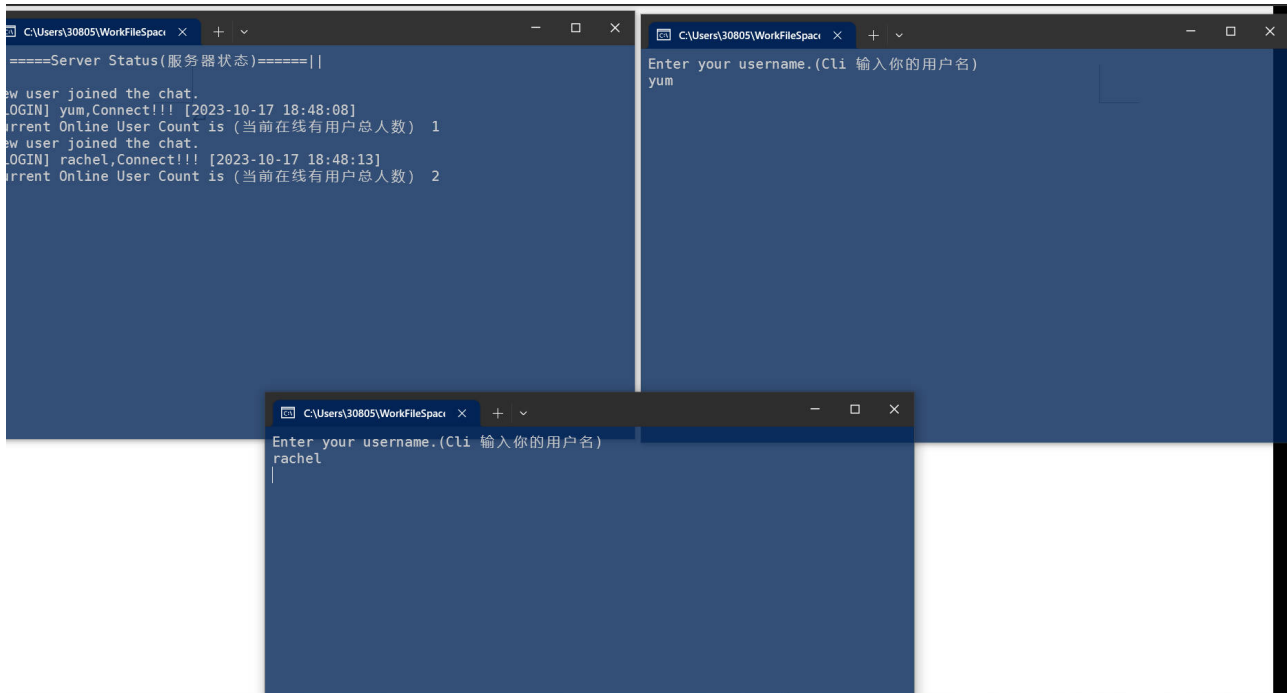
```

| TCPServer.h
|
|__src
|   ServerMain.cpp
|   TCPServer.cpp

```

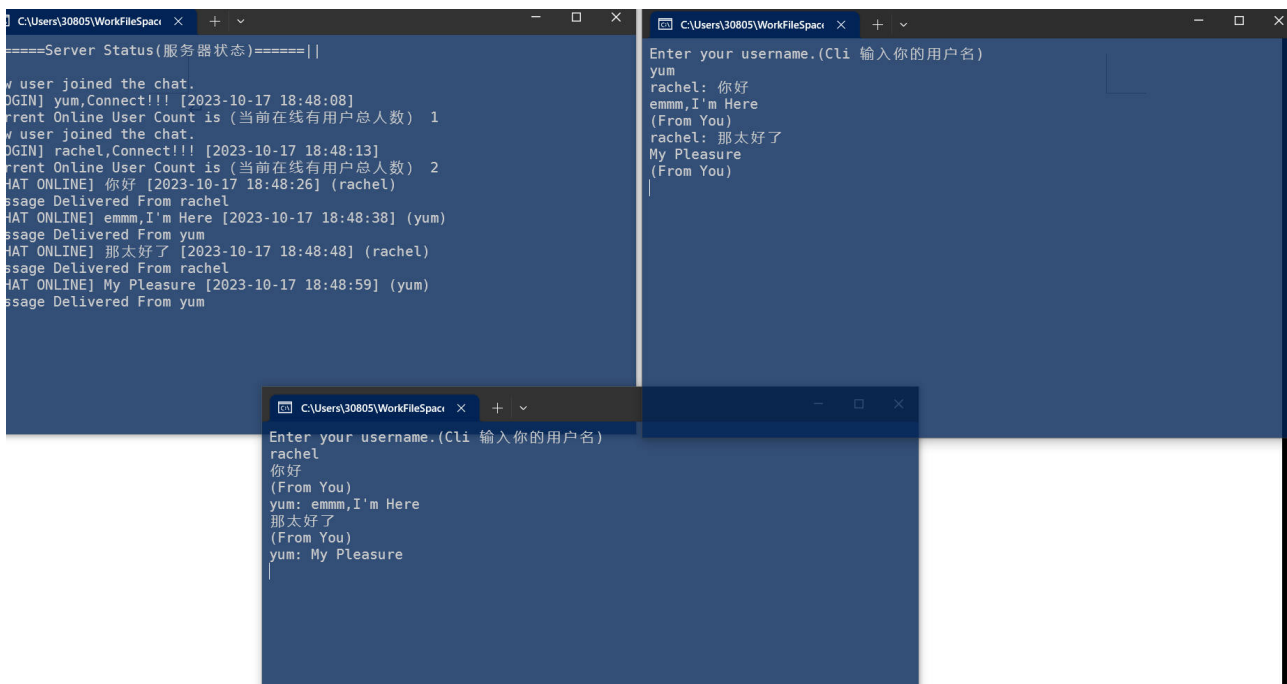
这里bin是二进制文件，多次点击即可运行Socket多线程多人聊天程序

首先系统登录界面:



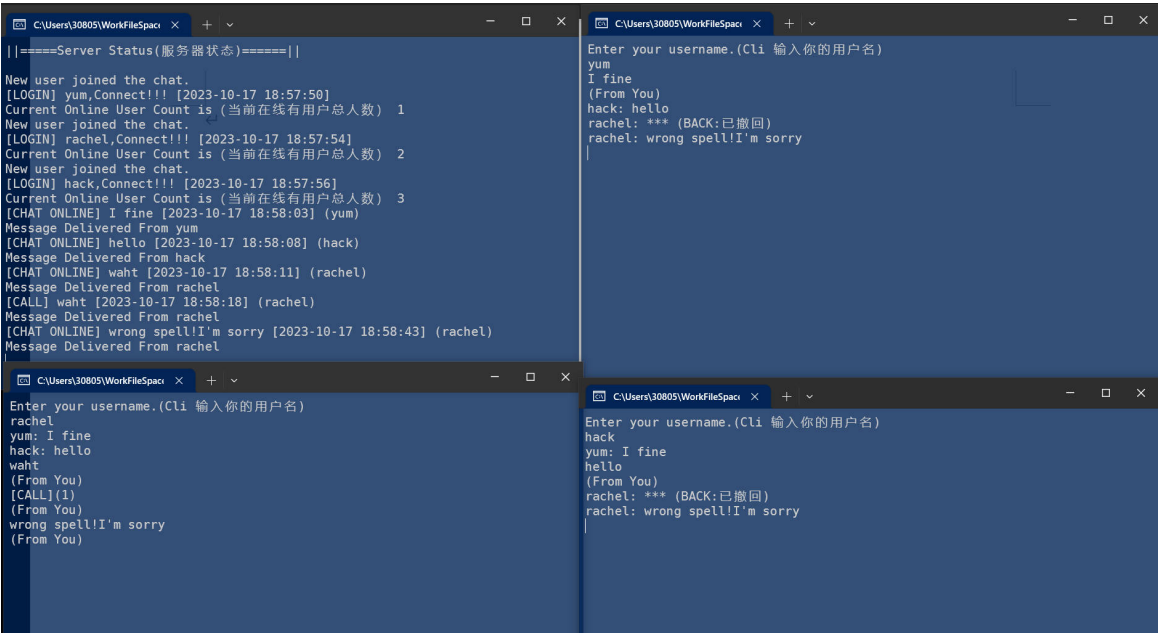
正常聊天通讯状态:

中英文



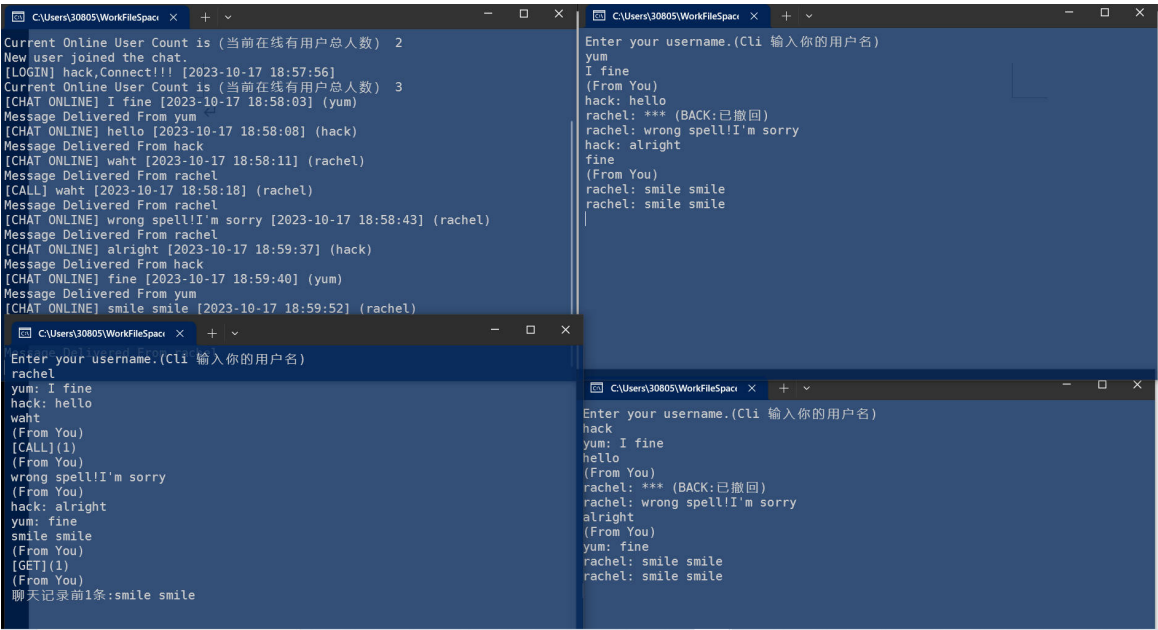
英文(消息撤回机制)

对于拼错的waht(**what**)及时撤回



英文(消息回放机制)

快速选中你需要的前几条历史消息



英文(用户自定义退出)


```
[CHAT ONLINE] I fine [2023-10-17 18:58:03] (yum)
Message Delivered From yum
[CHAT ONLINE] hello [2023-10-17 18:58:08] (hack)
Message Delivered From hack
[CHAT ONLINE] waht [2023-10-17 18:58:11] (rachel)
Message Delivered From rachel
[CALL] waht [2023-10-17 18:58:18] (rachel)
Message Delivered From rachel
[CHAT ONLINE] wrong spell!I'm sorry [2023-10-17 18:58:43] (rachel)
Message Delivered From rachel
[CHAT ONLINE] alright [2023-10-17 18:59:37] (hack)
Message Delivered From hack
[CHAT ONLINE] fine [2023-10-17 18:59:40] (yum)
Message Delivered From yum
[CHAT ONLINE] smile smile [2023-10-17 18:59:52] (rachel)
Message Delivered From rachel
[GET] waht [2023-10-17 19:00:06] (rachel)
Message Delivered From rachel
[LOGOUT] yum,GoodBye!!! [2023-10-17 19:00:30]

Enter your username.(Cli 输入你的用户名)
rachel
yum: I fine
hack: hello
waht
(From You)
[CALL](I)
(From You)
wrong spell!I'm sorry
(From You)
hack: alright
yum: fine
smile smile
(From You)
[GET](I)
(From You)
聊天记录前1条:smile smile
OFFLINE;BYE EVERYONE(From Server)[ONLINE TIME: 160 s]
hack: see you~

Enter your username.(Cli 输入你的用户名)
yum
I fine
(From You)
hack: hello
rachel: *** (BACK:已撤回)
rachel: wrong spell!I'm sorry
hack: alright
fine
(From You)
rachel: smile smile
rachel: smile smile
[OUT]
(From You)

Enter your username.(Cli 输入你的用户名)
hack
yum: I fine
hello
(From You)
rachel: *** (BACK:已撤回)
rachel: wrong spell!I'm sorry
alright
(From You)
yum: fine
rachel: smile smile
rachel: smile smile
OFFLINE;BYE EVERYONE(From Server)[ONLINE TIME: 160 s]
see you~
(From You)
```

5 程序分析和总结

优点:

实现多线程聊天通讯, 协议完整, 且每次都有定制化的响应附加信息, 能正确处理类似历史消息和消息撤回(英文)等功能。

缺点:

1.对用户TCP类的封装还不够,缺乏有效的安全认证机制。但不是实验重点

2, 不支持中文用户名和中文历史请求

3, 可以适当加入ftp等超文本通信机制, 丰富聊天